

```
use Illuminate\Support\Facades\Auth;
if (Auth::check()) {
    // The user is logged in...
}
```

UnAuthorised users

In case the validation is not successful, the user needs to be redirected to an appropriate web page e.g. ‘you did not enter the right username’ or ‘you have 3 more attempts remaining’. By default, on an unsuccessful attempt, a 401 error sign will show up or the user will be redirected to the ‘login’ route.

```
use Illuminate\Auth\AuthenticationException;
protected function unauthenticated($request, AuthenticationException $exception)
{
    return $request->expectsJson()
        ? response()->json(['message' => $exception->getMessage()], 401)
        : redirect()->guest(route('login'));
}
```

As seen in the above example, the action route can be re-specified by defining the ‘unauthenticated’ function in the app/Exceptions/Handler.php file. Another useful function that Laravel implements is ‘Login Throttling’ which basically means that the user will not be able to log in for one minute after several failed attempts. Throttling is unique to the user’s username / e-mail address and their IP address. It is in-built features like these that make Laravel such a reliable and developer friendly framework.

With that, we have a basic idea of how to implement an authentication procedure on our web application. We also have a rough idea of how this actually works. Most of the tools that you would be needing for these procedures are already shipped with the Laravel framework. As we have seen, all of these features are customisable and can be given new logic by using the right directories. Now we are going to study about another important feature of web applications and how Laravel helps us implement it.

Authorisation

While authentication is necessary in almost all web application, there is another important procedure to be followed called authorisation. Laravel